

GPU Hardware Discovery Guide

AMD & NVIDIA Discrete GPU Identification via DXGI and Python

1. The Problem: Integrated vs. Discrete

Modern Integrated GPUs (iGPUs) often reserve a small portion of System RAM (typically 128MB to 1GB) to report as "Dedicated Video Memory." Simple filters checking for memory > 0 often fail to distinguish these from high-performance discrete cards.

2. The Fix: Advanced Filtering Logic

The implementation utilizes `DXGI_ADAPTER_DESC2` to access hardware flags and a specific memory threshold.

- **Software Flag:** Filters out virtual drivers like the "Microsoft Basic Render Driver."
- **2GB Threshold:** Discrete GPUs almost always feature > **2GB** of VRAM, whereas APUs/iGPUs typically stay at or below 1GB.
- **Keyword Blacklist:** Catch-all for branding strings like "Graphics," "Integrated," or "UHD."

3. Technical Implementation (The "Runes")

This low-level approach talks directly to the Windows COM vtable. This is "Manual Labor Python" where we handle memory addresses directly.

```
# Key Filtering Indices
GetDesc2_Index = 10
QueryVideoMemoryInfo_Index = 14

# Memory Threshold
Threshold = 2 * 1024 * 1024 * 1024 # 2GB
```

4. Safety & Risks

Warning: Because `ctypes` bypasses Python's safety nets, incorrect vtable indices or missing `_release()` calls can cause immediate application crashes or memory leaks.

Component	Risk Level	Mitigation
VTable Logic	Moderate	Use standardized Microsoft indices (10, 12, 14).
COM Release	Low	Always call <code>_release(obj)</code> to free driver handles.
Memory Safety	None	Read-only operations do not impact hardware health.

5. Pythonic Logic Breakdown

If Windows were written in pure Python, `ctypes` logic would translate as follows:

VTable = List of Functions: Instead of `adapter.get_desc()`, we use `functions[10](adapter)`.

Structures = Slicing Bytes: We take a raw binary blob and slice it: `name = blob[0:256]`.

ByRef = Empty Boxes: Instead of `val = func()`, we give the driver an empty box and say "Fill this."